DESARROLLO DE APLICACIONES CON OBJETOS · 1º PARCIAL

# Guía del 1º parcial

El parcial real de la 4K3, los simulacros de la cátedra y los modelos de parcial, resueltos y verificados con sus tests

Parcial real 4K3 | 3 simulacros | 2 modelos de parcial | Todo pasa los tests de pytest | Receta paso a paso

**Qué hay en esta guía**

El **parcial real** de la 4K3 resuelto, los **tres simulacros** de la cátedra con su resolución y los **modelos de parcial** del repositorio de la cátedra (2026-guias). Todo con el enunciado tal cual y verificado con los tests de pytest que entrega la cátedra. Los que la cátedra no resolvió tienen una solución propuesta que también pasa sus tests.

Para verlo resuelto en video, paso a paso: **"Cómo resolver el parcial"** en la página de la materia (sección 1º Parcial).

## 0 Cómo es el parcial

Te dan tres cosas: el **enunciado**, un archivo **CSV** con los datos y un archivo de **tests** (`test_algo.py`, de pytest). Los tests son el contrato: dicen cómo se tienen que llamar los archivos, las clases y los métodos, y en qué orden van los parámetros. Si no coinciden, los tests no corren.

Siempre es el mismo esquema: una **clase base abstracta**, dos o tres **subclases** que calculan algo distinto (polimorfismo) y una **clase que guarda la colección** y responde lo que pide el enunciado. El tipo que viene en el CSV solo sirve para decidir qué subclase crear.

| Ejercicio | Clase base | Subclases (lo que cambia) | Colección y métodos |
|---|---|---|---|
| **Parcial 4K3 · Transporte de pasajeros** | Viaje | Urbano / Interurbano (importe +10 %) | Recaudación total, pasajeros por destino, viaje con más pasajeros |
| **La fábrica** | Mantenimiento | Preventivo (+ insumos) / Correctivo (+ técnico) | Suma de gastos, cantidad de caros, rotura más larga |
| **Hospital** | Atención | Médica / Farmacia (importe a cobrar) | Total de consultas, promedio entre dos valores, primera de un habitual |
| **Empresa de transporte** | Carga | Caja / Packing / Bidón (peso) | Camión: subir y bajar cargas, estados, listo para partir |
| **La inmobiliaria** | Inmueble | Casa / Departamento (alquiler) | Suma de alquileres, casas premium, propietario del más barato |
| **El supermercado** | Sucursal | Hiper / Super / Mini (resultado y rentabilidad) | Suma de ganancia, no rentables, local más rentable |

## Receta, paso a paso

1. **Leé el test antes que nada.** Anotá los nombres de los archivos (`from maquina import Maquina` → el archivo es `maquina.py`), de las clases, el orden de los parámetros del constructor y qué devuelve cada método (un número, un texto, un objeto o `None`).
2. **Clase base abstracta** (`ABC`) con los atributos comunes y el método que cambia según el tipo marcado con `@abstractmethod`.
3. **Subclases** que llaman a `super().__init__`, agregan sus atributos y redefinen el método. Nada de `if tipo == …` adentro de los cálculos: eso lo resuelve el polimorfismo.
4. **La clase de la colección** (Empresa, Máquina, Hospital…) con una lista y un método por cada pregunta: sumar (`sum`), contar con una condición (`len([… if …])`), el mayor o menor (`max(…, key=…)`) y `None` si no hay ninguno.
5. **El main**: leer el CSV (fijate si tiene línea de títulos), crear el objeto según el tipo, agregarlo a la colección e imprimir los resultados. El enunciado pide que los cálculos estén en la clase, no en el main.
6. **Correr** `python -m pytest` hasta que todo quede en verde.

### Errores que hacen fallar los tests

- **"Más de"** es `>`, no `>=`: los tests prueban justo el borde (por ejemplo, un gasto de exactamente $10.000 no es caro).
- El **orden de los parámetros** tiene que ser el del test, aunque el CSV venga en otro orden.
- Si el test compara `str(objeto)` o un texto, tiene que ser **idéntico** (espacios incluidos).
- Los nombres de archivo **exactamente** como los usa el test, mayúsculas incluidas. En Windows y Mac da igual, pero en Linux no: en los simulacros de Transporte y Mantenimiento los archivos de la cátedra empiezan con mayúscula y el test busca `camion.py` o `maquina.py` en minúscula para ubicar la carpeta.
- Con la colección vacía, devolver **0** o **None** según lo que espere el test, sin que se rompa.

# 1 El parcial real · 4K3 · Transporte de pasajeros

Parcial tomado | Solución propuesta (verificada con los tests)

### Enunciado (tal cual)

```
EJERCICIO 6 - TRANSPORTE DE PASAJEROS
Viaje urbano: pasajeros × precio base. Viaje interurbano: mismo importe +10%.
Leer viajes.csv. El tipo solo determina qué objeto crear.
1. Importe mediante polimorfismo. 2. Recaudación total. 3. Pasajeros por destino. 4. Viaje
con mayor cantidad de pasajeros.
Formato: tipo, codigo, conductor, destino, pasajeros, precio_base. Tipo 1=urbano;
2=interurbano.
```

Junto con el enunciado vienen `viajes.csv` (40 viajes, con línea de títulos) y este test, que fija el contrato: el archivo se llama `solucion.py` y las clases `Viaje`, `ViajeUrbano` y `ViajeInterurbano`, con `importe_recaudado()`.

test_viajes.py

```
import pytest
from solucion import Viaje,ViajeUrbano,ViajeInterurbano
def test_urbano():
 x=ViajeUrbano(1,"Ana","Cordoba",10,2000); assert isinstance(x,Viaje); assert
x.importe_recaudado()==pytest.approx(20000)
def test_interurbano():
 x=ViajeInterurbano(2,"Juan","Alta Gracia",10,2000); assert isinstance(x,Viaje); assert
x.importe_recaudado()==pytest.approx(22000)
```

### Cómo se piensa

- `Viaje` abstracta con los datos comunes; cada subclase calcula `importe_recaudado()`: urbano = pasajeros × precio base, interurbano = lo mismo × 1,10.
- El tipo del CSV (1 o 2) solo elige qué clase crear.
- Una clase `Empresa` guarda los viajes y responde: recaudación total (suma polimórfica), pasajeros por destino (diccionario de contadores) y el viaje con más pasajeros (`max` con `key`).

solucion.py

```python
import csv
from abc import ABC, abstractmethod


class Viaje(ABC):
    def __init__(self, codigo, conductor, destino, pasajeros, precio_base):
        self.codigo = codigo
        self.conductor = conductor
        self.destino = destino
        self.pasajeros = pasajeros
        self.precio_base = precio_base

    @abstractmethod
    def importe_recaudado(self):
        """Cada tipo de viaje calcula su importe (polimorfismo)."""
```

```
    def __str__(self):
        return (f"{type(self).__name__} {self.codigo} a {self.destino} · {self.conductor} · "
                f"{self.pasajeros} pasajeros · ${self.importe_recaudado():,.2f}")


class ViajeUrbano(Viaje):
    def importe_recaudado(self):
        return self.pasajeros * self.precio_base


class ViajeInterurbano(Viaje):
    def importe_recaudado(self):
        return self.pasajeros * self.precio_base * 1.10      # el mismo importe + 10 %


class Empresa:
    """Guarda los viajes y responde lo que pide el enunciado (no se hace en el main)."""

    def __init__(self):
        self.viajes = []

    def agregar(self, viaje):
        self.viajes.append(viaje)

    def recaudacion_total(self):
        return sum(v.importe_recaudado() for v in self.viajes)

    def pasajeros_por_destino(self):
        conteo = {}
        for v in self.viajes:
            conteo[v.destino] = conteo.get(v.destino, 0) + v.pasajeros
        return conteo

    def viaje_con_mas_pasajeros(self):
        return max(self.viajes, key=lambda v: v.pasajeros, default=None)


def leer_viajes(nombre_archivo, empresa):
    with open(nombre_archivo, encoding="utf-8") as archivo:
        for fila in csv.DictReader(archivo):                # la primera línea son los títulos
            clase = ViajeUrbano if fila["tipo"] == "1" else ViajeInterurbano   # el tipo solo
elige la clase
            empresa.agregar(clase(int(fila["codigo"]), fila["conductor"], fila["destino"],
                                  int(fila["pasajeros"]), float(fila["precio_base"])))


def main():
    empresa = Empresa()
    leer_viajes("viajes.csv", empresa)
    print("Viajes leídos:", len(empresa.viajes))
    print(f"Recaudación total: ${empresa.recaudacion_total():,.2f}")
    print("Pasajeros por destino:")
    for destino, cantidad in sorted(empresa.pasajeros_por_destino().items()):
        print(f"   {destino}: {cantidad}")
    print("Viaje con más pasajeros:", empresa.viaje_con_mas_pasajeros())


if __name__ == "__main__":
    main()
```

✓ Pasa los **2** tests de la cátedra ( `test_viajes.py` )

Salida al correr solucion.py

```
Viajes leídos: 40
Recaudación total: $3,510,745.00
Pasajeros por destino:
   Alta Gracia: 140
   Cordoba: 306
   Jesus Maria: 252
   Rio Cuarto: 189
   Villa Carlos Paz: 223
   Villa Maria: 210
Viaje con más pasajeros: ViajeInterurbano 6026 a Villa Carlos Paz · Persona 26 · 55 pasajeros · $169,400.00
```

# 2 Simulacro · La fábrica (mantenimientos)

Simulacro | Resolución de la cátedra

## Enunciado (tal cual)

### Examen Parcial

#### La fábrica

Una fábrica posee una máquina manufacturera a las que les realiza diversos mantenimientos para garantizar su correcto funcionamiento.

Cada vez que se le realiza un mantenimiento se registra el nombre del operario que lo realiza, la fecha en que se efectuó y el importe que se gastó en repuestos.

Existen dos tipos de mantenimiento, por un lado se realizan mantenimientos periódicos preventivos para reemplazar insumos (tales como lubricantes o repuestos de vida útil restringida) y para verificar su correcto funcionamiento. De estos mantenimientos preventivos se registra el importe gastado en insumos y el resultado del mismo, que puede ser 1 si la máquina se encuentra funcionando correctamente, 2 si se recomienda una revisión del servicio técnico del fabricante y 3 si se detecta alguna rotura.

Por otro lado, ocasionalmente la máquina queda impedida de funcionar y se requiere un mantenimiento correctivo. En estos casos se debe registrar la cantidad de horas de parada, es decir, el tiempo transcurrido entre que dejó de funcionar y que volvió a estar operativa. También debe registrarse el total que se abonó al técnico reparador.

Se necesita un programa que lea del archivo mantenimientos.csv la lista de todos los mantenimientos y los almacene en algún objeto (no en la función principal) que ofrezca los siguientes métodos:

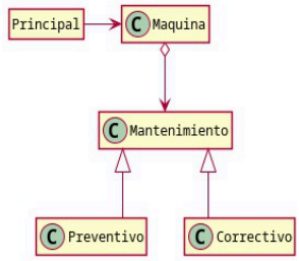
- Suma de gastos: que informe el total abonado por todo concepto en todos los mantenimientos registrados.
- Cantidad de mantenimientos caros: que informe el total de mantenimientos de cualquier tipo que hayan tenido un gasto total de más de $10.000
- Rotura más larga: que informe la fecha y el nombre del operario del mantenimiento correctivo de mayor duración.

La función principal debe ingresar los datos desde el archivo de texto y finalizar luego de imprimir el resultado de la ejecución de los tres métodos anteriores.

El archivo posee la siguiente estructura:

1. Tipo de mantenimiento: 1 para preventivo y 2 para correctivo
2. Fecha: una cadena
3. Operario: nombre del operario que realizó el mantenimiento
4. Importe de repuestos: número de tipo float
5. Si es un preventivo, resultado del mantenimiento (1, 2 o 3); si es un correctivo, cantidad de horas de parada.
6. Si es un preventivo, importe de los insumos; si es un correctivo, importe que cobró el técnico.

**Modelo tentativo**




La resolución de la cátedra separa una clase por archivo y valida cada dato con una clase `Validador` (en `utilities/`). Pasa sus 32 tests.

Mantenimiento.py

```python
# Imports
from abc import ABC, abstractmethod

from utilities.Validador import Validador

# Exports
__all__: list[str] = ["Mantenimiento"]


class Mantenimiento(ABC):
    def __init__(self, operario: str, fecha: str, importe_repuestos: float) -> None:
        self._operario: str = Validador.esCadenaNoVacia(operario)
        self._fecha: str = Validador.esCadenaNoVacia(fecha)
        self._importe_repuestos: float = Validador.esFlotantePositivo(importe_repuestos)

    @property
    def operario(self) -> str:
        return self._operario

    @operario.setter
    def operario(self, operario: str) -> None:
        self._operario = Validador.esCadenaNoVacia(operario)

    @property
    def fecha(self) -> str:
        return self._fecha

    @fecha.setter
    def fecha(self, fecha: str) -> None:
        self._fecha = Validador.esCadenaNoVacia(fecha)

    @property
    def importe_repuestos(self) -> float:
        return self._importe_repuestos

    @importe_repuestos.setter
    def importe_repuestos(self, importe_repuestos: float) -> None:
        self._importe_repuestos = Validador.esFlotantePositivo(importe_repuestos)

    @abstractmethod
    def gasto_total(self) -> float:
        pass

    def __str__(self) -> str:
        return (
            f"Mantenimiento del {self.fecha} - Operario: {self.operario} - "
            f"Repuestos: ${self.importe_repuestos:.2f} - "
            f"Gasto total: ${self.gasto_total():.2f}"
        )


if __name__ == "__main__":
    pass
```

Preventivo.py

```python
# ruff: noqa: RUF012

# Imports
from Mantenimiento import Mantenimiento
from utilities.Validador import Validador

# Exports
__all__: list[str] = ["Preventivo"]


class Preventivo(Mantenimiento):
    # Resultados del enunciado: 1 funciona correctamente, 2 revision recomendada, 3 rotura
    RESULTADOS: dict[int, str] = {
        1: "funciona correctamente",
        2: "revision recomendada",
        3: "rotura detectada",
    }

    def __init__(
        self,
        operario: str,
        fecha: str,
        importe_repuestos: float,
        importe_insumos: float,
        resultado: int,
    ) -> None:
        super().__init__(operario, fecha, importe_repuestos)
        self._importe_insumos: float = Validador.esFlotantePositivo(importe_insumos)
        self._resultado: int = self._validar_resultado(resultado)

    @staticmethod
    def _validar_resultado(resultado: int) -> int:
        if Validador.esEntero(resultado) not in Preventivo.RESULTADOS:
            raise ValueError("Error el resultado debe ser 1, 2 o 3")
        return resultado

    @property
    def importe_insumos(self) -> float:
        return self._importe_insumos

    @importe_insumos.setter
    def importe_insumos(self, importe_insumos: float) -> None:
        self._importe_insumos = Validador.esFlotantePositivo(importe_insumos)

    @property
    def resultado(self) -> int:
        return self._resultado

    @resultado.setter
    def resultado(self, resultado: int) -> None:
        self._resultado = self._validar_resultado(resultado)

    def gasto_total(self) -> float:
        return self.importe_repuestos + self.importe_insumos

    def __str__(self) -> str:
        return (
            f"Preventivo del {self.fecha} - Operario: {self.operario} - "
            f"Repuestos: ${self.importe_repuestos:.2f} - "
```

```
            f"Insumos: ${self.importe_insumos:.2f} - "
            f"Resultado: {self.RESULTADOS[self.resultado]} - "
            f"Gasto total: ${self.gasto_total():.2f}"
        )


if __name__ == "__main__":
    pass
```

Correctivo.py

```
# Imports
from Mantenimiento import Mantenimiento
from utilities.Validador import Validador

# Exports
__all__: list[str] = ["Correctivo"]


class Correctivo(Mantenimiento):
    def __init__(
        self,
        operario: str,
        fecha: str,
        importe_repuestos: float,
        horas_parada: int,
        importe_tecnico: float,
    ) -> None:
        super().__init__(operario, fecha, importe_repuestos)
        self._horas_parada: int = Validador.esEnteroPositivo(horas_parada)
        self._importe_tecnico: float = Validador.esFlotantePositivo(importe_tecnico)

    @property
    def horas_parada(self) -> int:
        return self._horas_parada

    @horas_parada.setter
    def horas_parada(self, horas_parada: int) -> None:
        self._horas_parada = Validador.esEnteroPositivo(horas_parada)

    @property
    def importe_tecnico(self) -> float:
        return self._importe_tecnico

    @importe_tecnico.setter
    def importe_tecnico(self, importe_tecnico: float) -> None:
        self._importe_tecnico = Validador.esFlotantePositivo(importe_tecnico)

    def gasto_total(self) -> float:
        return self.importe_repuestos + self.importe_tecnico

    def __str__(self) -> str:
        return (
            f"Correctivo del {self.fecha} - Operario: {self.operario} - "
            f"Repuestos: ${self.importe_repuestos:.2f} - "
            f"Horas de parada: {self.horas_parada} - "
            f"Tecnico: ${self.importe_tecnico:.2f} - "
            f"Gasto total: ${self.gasto_total():.2f}"
        )
```

```python
if __name__ == "__main__":
    pass
```

Maquina.py

```python
# pyright: reportUnnecessaryIsInstance=false

# Imports
from Correctivo import Correctivo
from Mantenimiento import Mantenimiento
from utilities.Validador import Validador

# Exports
__all__: list[str] = ["Maquina"]

GASTO_CARO: float = 10000.0


class Maquina:
    def __init__(self, nombre: str) -> None:
        self._nombre: str = Validador.esCadenaNoVacia(nombre)
        self._mantenimientos: list[Mantenimiento] = []

    @property
    def nombre(self) -> str:
        return self._nombre

    @nombre.setter
    def nombre(self, nombre: str) -> None:
        self._nombre = Validador.esCadenaNoVacia(nombre)

    @property
    def mantenimientos(self) -> list[Mantenimiento]:
        return self._mantenimientos

    @mantenimientos.setter
    def mantenimientos(self, mantenimientos: list[Mantenimiento]) -> None:
        if not isinstance(mantenimientos, list) or not all(
            isinstance(mantenimiento, Mantenimiento) for mantenimiento in mantenimientos
        ):
            raise TypeError("Error debe ser una lista de mantenimientos")
        self._mantenimientos = mantenimientos

    def add_mantenimiento(self, mantenimiento: Mantenimiento) -> None:
        if not isinstance(mantenimiento, Mantenimiento):
            raise TypeError(
                "Error el valor ingresado no corresponde a un mantenimiento"
            )
        self._mantenimientos.append(mantenimiento)

    def mantenimientos_correctivos(self) -> list[Correctivo]:
        return [
            mantenimiento
            for mantenimiento in self.mantenimientos
            if isinstance(mantenimiento, Correctivo)
        ]

    def suma_gastos(self) -> float:
        """Total abonado por todo concepto en todos los mantenimientos registrados."""
        return sum(mantenimiento.gasto_total() for mantenimiento in self.mantenimientos)
```

```
    def cantidad_mantenimientos_caros(self) -> int:
        """Cantidad de mantenimientos con un gasto total mayor a $10.000."""
        return len(
            [
                mantenimiento
                for mantenimiento in self.mantenimientos
                if mantenimiento.gasto_total() > GASTO_CARO
            ]
        )

    def rotura_mas_larga(self) -> Correctivo | None:
        """Mantenimiento correctivo de mayor duracion (mas horas de parada), o None."""
        correctivos: list[Correctivo] = self.mantenimientos_correctivos()
        if not correctivos:
            return None
        return max(correctivos, key=lambda mantenimiento: mantenimiento.horas_parada)

    def __str__(self) -> str:
        cadena: str = f"Maquina: {self.nombre}\n"
        for mantenimiento in self.mantenimientos:
            cadena += f"  {mantenimiento}\n"
        return cadena


if __name__ == "__main__":
    pass
```

main.py

```
"""Carga los mantenimientos de la maquina desde el archivo CSV de data/."""

import csv
from pathlib import Path

from Correctivo import Correctivo
from Mantenimiento import Mantenimiento
from Maquina import Maquina
from Preventivo import Preventivo

DIRECTORIO_DATOS: Path = Path(__file__).parent / "data"
ARCHIVO_MANTENIMIENTOS: Path = DIRECTORIO_DATOS / "mantenimientos.csv"

PREVENTIVO = 1
CORRECTIVO = 2


def leer_csv(ruta: Path) -> list[list[str]]:
    with open(ruta, newline="", encoding="utf-8") as archivo:
        return [fila for fila in csv.reader(archivo) if fila]


def crear_mantenimiento(fila: list[str]) -> Mantenimiento:
    tipo = int(fila[0])
    fecha: str = fila[1]
    operario: str = fila[2]
    importe_repuestos = float(fila[3])

    if tipo == PREVENTIVO:
        resultado = int(fila[4])
        importe_insumos = float(fila[5])
        return Preventivo(
```

```
            operario, fecha, importe_repuestos, importe_insumos, resultado
        )

    if tipo == CORRECTIVO:
        horas_parada = int(fila[4])
        importe_tecnico = float(fila[5])
        return Correctivo(
            operario, fecha, importe_repuestos, horas_parada, importe_tecnico
        )

    raise ValueError(
        f"Error el tipo de mantenimiento debe ser {PREVENTIVO} o {CORRECTIVO}"
    )


def cargar_mantenimientos(ruta: Path = ARCHIVO_MANTENIMIENTOS) -> list[Mantenimiento]:
    return [crear_mantenimiento(fila) for fila in leer_csv(ruta)]


def cargar_maquina(nombre: str = "Maquina Manufacturera") -> Maquina:
    maquina = Maquina(nombre)
    for mantenimiento in cargar_mantenimientos():
        maquina.add_mantenimiento(mantenimiento)
    return maquina


def main() -> None:
    maquina: Maquina = cargar_maquina()
    correctivos: list[Correctivo] = maquina.mantenimientos_correctivos()

    print(f"Maquina: {maquina.nombre}")
    print(f"Mantenimientos registrados: {len(maquina.mantenimientos)}")
    print(f"  Preventivos: {len(maquina.mantenimientos) - len(correctivos)}")
    print(f"  Correctivos: {len(correctivos)}")
    print()

    print(f"Suma de gastos: ${maquina.suma_gastos():.2f}")
    print(
        f"Cantidad de mantenimientos caros: {maquina.cantidad_mantenimientos_caros()}"
    )

    rotura: Correctivo | None = maquina.rotura_mas_larga()
    if rotura is not None:
        print(
            "Rotura mas larga: "
            f"fecha {rotura.fecha} - operario {rotura.operario} - "
            f"{rotura.horas_parada} horas de parada"
        )
    else:
        print("Rotura mas larga: no hay mantenimientos correctivos registrados")
    print()

    print("Detalle de los primeros 5 mantenimientos de cada tipo:")
    for mantenimiento in maquina.mantenimientos[:5]:
        print(f"  {mantenimiento}")
    for mantenimiento in correctivos[:5]:
        print(f"  {mantenimiento}")


if __name__ == "__main__":
    main()
```

utilities/Validador.py

```python
__all__: list[str] = ["Validador"]


class Validador:
    @staticmethod
    def esFlotante(numero: float) -> float:
        if isinstance(numero, bool) or type(numero) not in (int, float):
            raise TypeError("Error debe ser un numero flotante")
        return float(numero)

    @staticmethod
    def esEntero(numero: int) -> int:
        if isinstance(numero, bool) or type(numero) != int:
            raise TypeError("Error debe ser un numero entero")
        return numero

    @staticmethod
    def esFlotantePositivo(numero: float) -> float:
        numero = Validador.esFlotante(numero)
        if numero < 0:
            raise TypeError("Error debe ser un numero flotante positivo")
        return numero

    @staticmethod
    def esEnteroPositivo(numero: int) -> int:
        if Validador.esEntero(numero) < 0:
            raise TypeError("Error debe ser un numero entero positivo")
        return numero

    @staticmethod
    def esCadenaNoVacia(cadena: str) -> str:
        if type(cadena) != str or not cadena.strip():
            raise TypeError("Error debe ser una cadena no vacia")
        return cadena.strip()
```

✓ Pasa los **32** tests de la cátedra (`tests/autocontenido/test_mantenimiento_autocontenido.py`) · en Linux, con una copia en minúscula de `Maquina.py` y `Mantenimiento.py`

Salida al correrlo

```
Maquina: Maquina Manufacturera
Mantenimientos registrados: 99
  Preventivos: 48
  Correctivos: 51

Suma de gastos: $3023600.00
Cantidad de mantenimientos caros: 88
Rotura mas larga: fecha 10/4 - operario PILAR - 29 horas de parada

Detalle de los primeros 5 mantenimientos de cada tipo:
  Preventivo del 4/7 - Operario: MARIA ROSARIO - Repuestos: $15300.00 - Insumos: $4700.00 -
Resultado: revision recomendada - Gasto total: $20000.00
  Correctivo del 4/7 - Operario: JULIAN - Repuestos: $11100.00 - Horas de parada: 1 - Tecnico:
$1400.00 - Gasto total: $12500.00
  Preventivo del 8/1 - Operario: JULIO - Repuestos: $16400.00 - Insumos: $900.00 - Resultado:
funciona correctamente - Gasto total: $17300.00
  Correctivo del 21/1 - Operario: ANA BELEN - Repuestos: $400.00 - Horas de parada: 7 - Tecnico:
$15300.00 - Gasto total: $15700.00
  Correctivo del 14/7 - Operario: LORENZO - Repuestos: $27500.00 - Horas de parada: 27 - Tecnico:
$36600.00 - Gasto total: $64100.00
  Correctivo del 4/7 - Operario: JULIAN - Repuestos: $11100.00 - Horas de parada: 1 - Tecnico:
$1400.00 - Gasto total: $12500.00
  Correctivo del 21/1 - Operario: ANA BELEN - Repuestos: $400.00 - Horas de parada: 7 - Tecnico:
$15300.00 - Gasto total: $15700.00
  Correctivo del 14/7 - Operario: LORENZO - Repuestos: $27500.00 - Horas de parada: 27 - Tecnico:
$36600.00 - Gasto total: $64100.00
  Correctivo del 6/9 - Operario: ESTHER - Repuestos: $26800.00 - Horas de parada: 1 - Tecnico:
$29900.00 - Gasto total: $56700.00
  Correctivo del 19/1 - Operario: MARIANO - Repuestos: $25500.00 - Horas de parada: 11 - Tecnico:
$43400.00 - Gasto total: $68900.00
```

**Para tener en cuenta**

- En el `main`, el bloque “primeros 5 de cada tipo” muestra los primeros 5 de **todos** (mezclados) y después 5 correctivos: para los preventivos habría que filtrarlos.

## El mismo ejercicio, con el contrato del repositorio

En el repositorio 2026-guias este ejercicio es el modelo de parcial **“La fábrica”**, con otro test (`test_fabrica.py`, 12 pruebas) y otro contrato: `Maquina()` sin nombre, `Preventivo(fecha, operario, repuestos, resultado, insumos)` y `rotura_mas_larga()` que devuelve el texto `"fecha OPERARIO"` o `None`. Mismo problema, distinto test: por eso hay que leer el test primero. Esta solución lo cumple y da los mismos resultados que la de la cátedra:

mantenimiento.py

```python
from abc import ABC, abstractmethod


class Mantenimiento(ABC):
    def __init__(self, fecha, operario, repuestos):
        self.fecha = fecha
        self.operario = operario
        self.repuestos = repuestos

    @abstractmethod
    def gasto_total(self):
        """Repuestos más lo propio de cada tipo."""

    def __str__(self):
        return f"{type(self).__name__} del {self.fecha} · {self.operario} ·
${self.gasto_total():,.2f}"
```

preventivo.py

```python
from mantenimiento import Mantenimiento


class Preventivo(Mantenimiento):
    def __init__(self, fecha, operario, repuestos, resultado, insumos):
        super().__init__(fecha, operario, repuestos)
        self.resultado = resultado          # 1 funciona, 2 revisión del fabricante, 3 rotura
        self.insumos = insumos

    def gasto_total(self):
        return self.repuestos + self.insumos
```

correctivo.py

```python
from mantenimiento import Mantenimiento


class Correctivo(Mantenimiento):
    def __init__(self, fecha, operario, repuestos, horas_parada, tecnico):
        super().__init__(fecha, operario, repuestos)
        self.horas_parada = horas_parada
        self.tecnico = tecnico

    def gasto_total(self):
        return self.repuestos + self.tecnico
```

maquina.py

```python
from correctivo import Correctivo


class Maquina:
    def __init__(self):
        self.mantenimientos = []

    def agregar_mantenimiento(self, mantenimiento):
        self.mantenimientos.append(mantenimiento)

    def suma_gastos(self):
        return sum(m.gasto_total() for m in self.mantenimientos)

    def cantidad_mantenimientos_caros(self):
        return len([m for m in self.mantenimientos if m.gasto_total() > 10000])     # más de
$10.000

    def rotura_mas_larga(self):
        correctivos = [m for m in self.mantenimientos if isinstance(m, Correctivo)]
        if not correctivos:
            return None
        mayor = max(correctivos, key=lambda m: m.horas_parada)
        return f"{mayor.fecha} {mayor.operario}"
```

main.py

```python
from correctivo import Correctivo
from maquina import Maquina
from preventivo import Preventivo


def leer_mantenimientos(nombre_archivo, maquina):
    with open(nombre_archivo, encoding="utf-8") as archivo:
        for linea in archivo:
            if not linea.strip():
                continue
            tipo, fecha, operario, repuestos, dato5, dato6 = linea.strip().split(",")
            if tipo == "1":
                maquina.agregar_mantenimiento(Preventivo(fecha, operario, float(repuestos),
int(dato5), float(dato6)))
            else:
                maquina.agregar_mantenimiento(Correctivo(fecha, operario, float(repuestos),
int(dato5), float(dato6)))


def main():
    maquina = Maquina()
    leer_mantenimientos("mantenimientos.csv", maquina)
    print("Mantenimientos leídos:", len(maquina.mantenimientos))
    print(f"Suma de gastos: ${maquina.suma_gastos():,.2f}")
    print("Mantenimientos caros (más de $10.000):", maquina.cantidad_mantenimientos_caros())
    print("Rotura más larga:", maquina.rotura_mas_larga())


if __name__ == "__main__":
    main()
```

✓ Pasa los **12** tests de la cátedra ( `test_fabrica.py` )

Salida al correrlo

```
Mantenimientos leídos: 99
Suma de gastos: $3,023,600.00
Mantenimientos caros (más de $10.000): 88
Rotura más larga: 10/4 PILAR
```

# 3 Simulacro · Hospital

Simulacro | Resolución de la cátedra

**Enunciado (tal cual)**

Un hospital de nuestra ciudad necesita conocer cierta información respecto a los importes cobrados por las distintas atenciones que realiza,y requiere de un sistema orientado a objetos que le dé soporte a ello. Los datos relevantes son los siguientes:

Del Hospital sólo interesa registrar la razón social; y una colección con todas las atenciones realizadas.

Cada atención tiene los siguientes datos: un código numérico que identifica a cada atención; y un valor numérico que representa el tipode cobro (1: “efectivo”; 2: “tarjeta de crédito”). Las atenciones pueden ser médicas o de farmacia:

Una atención médica agrega los siguientes datos: el paciente atendido; y el importe de la consulta.

Una atención de farmacia agregan los siguientes datos: el importe total de los medicamentos vendidos en dicha atención; y un cupón de descuento que especifica el monto de descuento que se aplicaría sobre el importe total de los medicamentos; en caso que el cupón de descuento sea igual que 0, no se realizará ningún descuento; cabe aclarar que el cupón de descuento debe ser 0 o positivo.

Del paciente asociado a la atención médica se registran: su nombre; el síntoma que prevalece (1: “corazon”, 2: “pulmon”, 3: “otras”); y un valor booleano que representa si el paciente es habitual (true) o no (false) del hospital.

De cada clase de atención se debe calcular un importeACobrar en pesos, que representa lo que el hospital cobra al paciente por la atención brindada. Este cálculo dependerá del tipo de cobro de la atención y de la clase de atención brindada:

El importe total de las atenciones médicas se calcula a partir del importe de la consulta. En caso que el paciente sea “habitual” se leaplicará un descuento del 25%. Por último, si el tipo de cobro es igual a 2 (“tarjeta de crédito”) se le incrementa al anterior importe un20% más; en caso que el cobro sea igual a 1 (“efectivo”) se le realiza un descuento del 10% sobre el anterior importe.

El importe total de las atenciones de farmacia se calcula a partir del importe total de los medicamentos vendidos. Luego se le realiza el descuento del cupón de descuento cuyo importe está especificado. Por último, si el tipo de cobro es igual a 2 (“tarjeta de crédito”) se leincrementa al anterior importe un 30% más; en caso que el cobro sea igual a 1 (“efectivo”) se le realiza un descuento del 5% sobre elanterior importe.

Con lo expuesto anteriormente, usted deberá implementar:

Todas las clases del modelo presentado.

Los siguientes requerimientos de métodos:

- Para las clases Atención, Médica, Farmacia y Paciente implementar su constructor, acceso y modificación y toString.
- Para la clase Hospital implementar el método addAtención que agrega una atención a la colección.
- Definir e implementar los métodos importeACobrar de las atenciones, que calcule y devuelva el importe a cobrar por la atención de acuerdo a los criterios anteriormente especificados.
- En clase Hospital se requiere la implementación de los siguientes métodos:

    - importe_total_atencion_consulta: debe calcular la suma de los importes de las consultas de las atenciones médicas.
    - importe_promedio_atenciones: debe calcular el promedio de los importes a cobrar por aquellas atenciones médicas cuyo importe a cobrar se encuentre entre dos valores recibidos como parámetros.
    - codigo_primera_atencion_habitual: debe retornar el código de la primera atención médica que se haya registrado para un paciente habitual, o 0 si no existe ninguna.
- Los valores de los objetos que deben ser instanciados se encuentran en la carpeta `data/`

Los datos vienen en tres CSV (`data/pacientes.csv`, `atenciones_medicas.csv` y `atenciones_farmacia.csv`). La resolución de la cátedra pone las tres atenciones en `atencion.py`:

atencion.py

```python
# Imports
from abc import ABC, abstractmethod

from paciente import Paciente
from utilities import Validador

# Exports
__all__ = ["Atencion", "AtencionMedica", "AtencionFarmacia"]


class Atencion(ABC):
    # Tipos de cobro del enunciado: 1 efectivo, 2 tarjeta de credito
    TIPOS_DE_COBRO = {1: "efectivo", 2: "tarjeta de credito"}

    def __init__(self, codigo: int, tipoDeCobro: int) -> None:
        self._codigo = Validador.esEnteroPositivo(codigo)
        self._tipoDeCobro = self._validarTipoDeCobro(tipoDeCobro)

    @staticmethod
    def _validarTipoDeCobro(tipoDeCobro: int) -> int:
        if Validador.esEntero(tipoDeCobro) not in Atencion.TIPOS_DE_COBRO:
            raise ValueError("Error el tipo de cobro debe ser 1 o 2")
        return tipoDeCobro

    @property
    def codigo(self) -> int:
        return self._codigo

    @codigo.setter
    def codigo(self, codigo: int):
        self._codigo = Validador.esEnteroPositivo(codigo)

    @property
    def tipoDeCobro(self) -> int:
        return self._tipoDeCobro

    @tipoDeCobro.setter
    def tipoDeCobro(self, tipoDeCobro: int) -> None:
        self._tipoDeCobro = self._validarTipoDeCobro(tipoDeCobro)

    @abstractmethod
    def importeACobrar(self) -> float:
        pass
```

```
    def __str__(self) -> str:
        return (
            f"Atencion {self.codigo} - "
            f"Cobro: {self.TIPOS_DE_COBRO[self.tipoDeCobro]} - "
            f"Importe a cobrar: ${self.importeACobrar():.2f}"
        )


class AtencionMedica(Atencion):
    def __init__(
        self, codigo: int, tipoDeCobro: int, paciente: Paciente, importe: float
    ) -> None:
        super().__init__(codigo, tipoDeCobro)
        self._paciente = self._validarPaciente(paciente)
        self._importe = Validador.esFlotantePositivo(importe)

    @staticmethod
    def _validarPaciente(paciente: Paciente) -> Paciente:
        if not isinstance(paciente, Paciente):
            raise ValueError("Error el valor ingresado no corresponde a un paciente")
        return paciente

    @property
    def paciente(self) -> Paciente:
        return self._paciente

    @paciente.setter
    def paciente(self, paciente: Paciente):
        self._paciente = self._validarPaciente(paciente)

    @property
    def importe(self) -> float:
        return self._importe

    @importe.setter
    def importe(self, importe: float):
        self._importe = Validador.esFlotantePositivo(importe)

    def importeACobrar(self) -> float:
        valor = self.importe
        if self.paciente.habitual:
            valor = valor * 0.75
        match self.tipoDeCobro:
            case 1:
                valor = valor * 0.9
            case 2:
                valor = valor * 1.2
        return valor

    def esPacienteHabitual(self) -> bool:
        return self.paciente.habitual

    def __str__(self) -> str:
        return (
            f"Atencion Medica {self.codigo} - "
            f"Cobro: {self.TIPOS_DE_COBRO[self.tipoDeCobro]} - "
            f"Consulta: ${self.importe:.2f} - {self.paciente} - "
            f"Importe a cobrar: ${self.importeACobrar():.2f}"
        )
```

```python
class AtencionFarmacia(Atencion):
    def __init__(
        self,
        codigo: int,
        tipoDeCobro: int,
        importeTotal: float,
        descuento: float,
    ) -> None:
        super().__init__(codigo, tipoDeCobro)
        self._importeTotal = Validador.esFlotantePositivo(importeTotal)
        self._descuento = Validador.esFlotantePositivo(descuento)

    @property
    def importeTotal(self) -> float:
        return self._importeTotal

    @importeTotal.setter
    def importeTotal(self, importeTotal: float) -> None:
        self._importeTotal = Validador.esFlotantePositivo(importeTotal)

    @property
    def descuento(self) -> float:
        return self._descuento

    @descuento.setter
    def descuento(self, descuento: float) -> None:
        self._descuento = Validador.esFlotantePositivo(descuento)

    def importeACobrar(self) -> float:
        # El cupon es un monto fijo a restar; si es 0 no se aplica descuento.
        valor = max(self.importeTotal - self.descuento, 0.0)
        match self.tipoDeCobro:
            case 1:
                valor = valor * 0.95
            case 2:
                valor = valor * 1.3
        return valor

    def __str__(self) -> str:
        return (
            f"Atencion Farmacia {self.codigo} - "
            f"Cobro: {self.TIPOS_DE_COBRO[self.tipoDeCobro]} - "
            f"Medicamentos: ${self.importeTotal:.2f} - Cupon: ${self.descuento:.2f} - "
            f"Importe a cobrar: ${self.importeACobrar():.2f}"
        )


if __name__ == "__main__":
    pass
# print("Hola")
```

paciente.py

```python
from utilities import Validador

# Exports
__all__ = ["Paciente"]


class Paciente:
    # Sintomas del enunciado: 1 corazon, 2 pulmon, 3 otras
```

```
    SINTOMAS = {1: "corazon", 2: "pulmon", 3: "otras"}

    def __init__(self, nombre: str, sintoma: int, habitual: bool = False) -> None:
        self._nombre = Validador.esCadenaNoVacia(nombre)
        self._sintoma = self._validarSintoma(sintoma)
        self._habitual = self._validarHabitual(habitual)

    @staticmethod
    def _validarSintoma(sintoma: int) -> int:
        if Validador.esEntero(sintoma) not in Paciente.SINTOMAS:
            raise ValueError("Error el sintoma debe ser 1, 2 o 3")
        return sintoma

    @staticmethod
    def _validarHabitual(habitual: bool) -> bool:
        if not isinstance(habitual, bool):
            raise ValueError("Error habitual debe ser un booleano")
        return habitual

    @property
    def nombre(self) -> str:
        return self._nombre

    @nombre.setter
    def nombre(self, nombre: str) -> None:

        self._nombre = Validador.esCadenaNoVacia(nombre)

    @property
    def sintoma(self) -> int:
        return self._sintoma

    @sintoma.setter
    def sintoma(self, sintoma: int) -> None:
        self._sintoma = self._validarSintoma(sintoma)

    @property
    def habitual(self) -> bool:
        return self._habitual

    @habitual.setter
    def habitual(self, habitual: bool) -> None:
        self._habitual = self._validarHabitual(habitual)

    def __str__(self) -> str:
        return (
            f"Paciente: {self.nombre} - Sintoma: {self.SINTOMAS[self.sintoma]} - "
            f"Habitual: {'si' if self.habitual else 'no'}"
        )


if __name__ == "__main__":
    persona = Paciente("Huenu", 1, True)

    print(persona)
```

hospital.py

```python
from functools import reduce

from atencion import Atencion, AtencionMedica
from utilities import Validador

# Exports
__all__ = ["Hospital"]


class Hospital:
    def __init__(self, razonSocial: str):
        self._razonSocial = Validador.esCadenaNoVacia(razonSocial)
        self._atencionesRealizadas: list[Atencion] = []

    @property
    def razonSocial(self) -> str:
        return self._razonSocial

    @razonSocial.setter
    def razonSocial(self, razonSocial: str) -> None:
        self._razonSocial = Validador.esCadenaNoVacia(razonSocial)

    @property
    def atencionesRealizadas(self) -> list[Atencion]:
        return self._atencionesRealizadas

    @atencionesRealizadas.setter
    def atencionesRealizadas(self, atencionesRealizadas: list) -> None:
        if not isinstance(atencionesRealizadas, list) or not all(
            isinstance(atencion, Atencion) for atencion in atencionesRealizadas
        ):
            raise ValueError("Error debe ser una lista de atenciones")
        self._atencionesRealizadas = atencionesRealizadas

    def addAtencion(self, atencion: Atencion) -> None:
        if not isinstance(atencion, Atencion):
            raise ValueError("Error el valor ingresado no corresponde a una atencion")
        self._atencionesRealizadas.append(atencion)

    def atencionesMedicas(self) -> list[AtencionMedica]:
        return list(
            filter(
                lambda atencion: isinstance(atencion, AtencionMedica),
                self.atencionesRealizadas,
            )
        )

    def importe_total_atencion_consulta(self) -> float:
        """Suma de los importes de las consultas de las atenciones medicas."""
        return reduce(
            lambda acumulador, atencion: acumulador + atencion.importe,
            self.atencionesMedicas(),
            0.0,
        )

    def importe_promedio_atenciones(self, minimo: float, maximo: float) -> float:
        """Promedio de los importes a cobrar de las atenciones medicas cuyo
        importe a cobrar se encuentra entre minimo y maximo."""
        minimo = Validador.esFlotantePositivo(minimo)
```

```
        maximo = Validador.esFlotantePositivo(maximo)

        importes = list(
            map(
                lambda atencion: atencion.importeACobrar(),
                filter(
                    lambda atencion: minimo <= atencion.importeACobrar() <= maximo,
                    self.atencionesMedicas(),
                ),
            )
        )
        return sum(importes) / len(importes) if importes else 0.0

    def codigo_primera_atencion_habitual(self) -> int:
        """Codigo de la primera atencion medica de un paciente habitual, o 0."""
        for atencion in self.atencionesMedicas():
            if atencion.esPacienteHabitual():
                return atencion.codigo
        return 0

    def __str__(self) -> str:
        cadena = f"Hospital - Razon Social: {self.razonSocial}\n"
        for atencion in self.atencionesRealizadas:
            cadena += f"  {atencion}\n"
        return cadena


if __name__ == "__main__":
    pass
```

main.py

```
"""Carga las atenciones del hospital desde los archivos CSV de data/."""

import csv
from pathlib import Path

from atencion import AtencionFarmacia, AtencionMedica
from hospital import Hospital
from paciente import Paciente

DIRECTORIO_DATOS = Path(__file__).parent / "data"
ARCHIVO_PACIENTES = DIRECTORIO_DATOS / "pacientes.csv"
ARCHIVO_MEDICAS = DIRECTORIO_DATOS / "atenciones_medicas.csv"
ARCHIVO_FARMACIA = DIRECTORIO_DATOS / "atenciones_farmacia.csv"


def leerCsv(ruta: Path) -> list[dict]:
    with open(ruta, newline="", encoding="utf-8") as archivo:
        return list(csv.DictReader(archivo))


def aBooleano(valor: str) -> bool:
    return valor.strip().lower() in ("true", "1", "si", "sí")


def cargarPacientes(ruta: Path = ARCHIVO_PACIENTES) -> dict[int, Paciente]:
    """Devuelve los pacientes indexados por el codigo de la atencion medica
    con la que se realizo la atencion."""
    pacientes: dict[int, Paciente] = {}
    for fila in leerCsv(ruta):
```

```
        codigoAtencion = int(fila["codigo_atencion"])
        pacientes[codigoAtencion] = Paciente(
            nombre=fila["nombre"],
            sintoma=int(fila["sintoma"]),
            habitual=aBooleano(fila["habitual"]),
        )
    return pacientes


def cargarAtencionesMedicas(
    pacientes: dict[int, Paciente], ruta: Path = ARCHIVO_MEDICAS
) -> list[AtencionMedica]:
    atenciones = []
    for fila in leerCsv(ruta):
        codigo = int(fila["codigo"])
        paciente = pacientes.get(codigo)
        if paciente is None:
            raise ValueError(f"No hay paciente para la atencion medica {codigo}")
        atenciones.append(
            AtencionMedica(
                codigo=codigo,
                tipoDeCobro=int(fila["tipo_cobro"]),
                paciente=paciente,
                importe=float(fila["importe_consulta"]),
            )
        )
    return atenciones


def cargarAtencionesFarmacia(ruta: Path = ARCHIVO_FARMACIA) -> list[AtencionFarmacia]:
    return [
        AtencionFarmacia(
            codigo=int(fila["codigo"]),
            tipoDeCobro=int(fila["tipo_cobro"]),
            importeTotal=float(fila["importe_total"]),
            descuento=float(fila["cupon_descuento"]),
        )
        for fila in leerCsv(ruta)
    ]


def cargarHospital(razonSocial: str = "Hospital San Roque") -> Hospital:
    hospital = Hospital(razonSocial)
    pacientes = cargarPacientes()

    for atencion in cargarAtencionesMedicas(pacientes):
        hospital.addAtencion(atencion)

    for atencion in cargarAtencionesFarmacia():
        hospital.addAtencion(atencion)

    return hospital


def main() -> None:
    hospital = cargarHospital()
    medicas = hospital.atencionesMedicas()

    print(f"Hospital: {hospital.razonSocial}")
    print(f"Atenciones realizadas: {len(hospital.atencionesRealizadas)}")
    print(f"  Medicas : {len(medicas)}")
```

```
    print(f"  Farmacia: {len(hospital.atencionesRealizadas) - len(medicas)}")
    print()

    print(
        "Importe total de las consultas medicas: "
        f"${hospital.importe_total_atencion_consulta():.2f}"
    )

    minimo, maximo = 10000.0, 30000.0
    print(
        f"Importe promedio de atenciones medicas entre ${minimo:.2f} y ${maximo:.2f}: "
        f"${hospital.importe_promedio_atenciones(minimo, maximo):.2f}"
    )

    print(
        "Codigo de la primera atencion de un paciente habitual: "
        f"{hospital.codigo_primera_atencion_habitual()}"
    )
    print()

    print("Detalle de las primeras 5 atenciones de cada tipo:")
    for atencion in medicas[:5]:
        print(f"  {atencion}")
    for atencion in hospital.atencionesRealizadas[-30:][:5]:
        print(f"  {atencion}")


if __name__ == "__main__":
    main()
```

utilities/validadores.py

```python
__all__ = ["Validador"]


class Validador:

    @staticmethod
    def esFlotante(numero: float) -> float:
        if isinstance(numero, bool) or not isinstance(numero, (int, float)):
            raise ValueError("Error debe ser un numero flotante")
        return float(numero)

    @staticmethod
    def esEntero(numero: int) -> int:
        if isinstance(numero, bool) or type(numero) != int:
            raise ValueError("Error debe ser un numero entero")
        return numero

    @staticmethod
    def esFlotantePositivo(numero: float) -> float:
        numero = Validador.esFlotante(numero)
        if numero < 0:
            raise ValueError("Error debe ser un numero flotante positivo")
        return numero

    @staticmethod
    def esEnteroPositivo(numero: int) -> int:
        if Validador.esEntero(numero) < 0:
            raise ValueError("Error debe ser un numero entero positivo")
        return numero

    @staticmethod
    def esCadenaNoVacia(cadena: str) -> str:
        if type(cadena) != str or not cadena.strip():
            raise ValueError("Error debe ser una cadena no vacia")
        return cadena.strip()
```

✓ Pasa los **49** tests de la cátedra ( `tests/autocontenido/test_hospital_autocontenido.py` )

Salida al correrlo

```
Hospital: Hospital San Roque
Atenciones realizadas: 60
  Medicas : 30
  Farmacia: 30

Importe total de las consultas medicas: $864634.55
Importe promedio de atenciones medicas entre $10000.00 y $30000.00: $21958.64
Codigo de la primera atencion de un paciente habitual: 1

Detalle de las primeras 5 atenciones de cada tipo:
  Atencion Medica 1 - Cobro: efectivo - Consulta: $17717.06 - Paciente: Ana Diaz - Sintoma: otras
- Habitual: si - Importe a cobrar: $11959.02
  Atencion Medica 2 - Cobro: efectivo - Consulta: $43318.92 - Paciente: Luis Perez - Sintoma:
otras - Habitual: no - Importe a cobrar: $38987.03
  Atencion Medica 3 - Cobro: tarjeta de credito - Consulta: $36920.66 - Paciente: Marta Gomez -
Sintoma: otras - Habitual: no - Importe a cobrar: $44304.79
  Atencion Medica 4 - Cobro: efectivo - Consulta: $3612.80 - Paciente: Jorge Ruiz - Sintoma:
corazon - Habitual: si - Importe a cobrar: $2438.64
  Atencion Medica 5 - Cobro: tarjeta de credito - Consulta: $39845.88 - Paciente: Sofia Luna -
Sintoma: pulmon - Habitual: si - Importe a cobrar: $35861.29
  Atencion Farmacia 101 - Cobro: efectivo - Medicamentos: $38296.13 - Cupon: $0.00 - Importe a
cobrar: $36381.32
  Atencion Farmacia 102 - Cobro: tarjeta de credito - Medicamentos: $38961.34 - Cupon: $11187.13
- Importe a cobrar: $36106.47
  Atencion Farmacia 103 - Cobro: tarjeta de credito - Medicamentos: $10609.46 - Cupon: $0.00 -
Importe a cobrar: $13792.30
  Atencion Farmacia 104 - Cobro: tarjeta de credito - Medicamentos: $43137.20 - Cupon: $7154.17 -
Importe a cobrar: $46777.94
  Atencion Farmacia 105 - Cobro: tarjeta de credito - Medicamentos: $36946.42 - Cupon: $9734.25 -
Importe a cobrar: $35375.82
```

# 4 Simulacro · Empresa de transporte de cargas

Simulacro | Resolución de la cátedra

## Enunciado (tal cual)

### Caso de estudio

#### Empresa de transporte de cargas

Una empresa de transporte de cargas necesita un software que la ayude a organizarse con la carga de los camiones que maneja. La empresa puede recibir packings, cajas sueltas y bidones que transportan líquido.

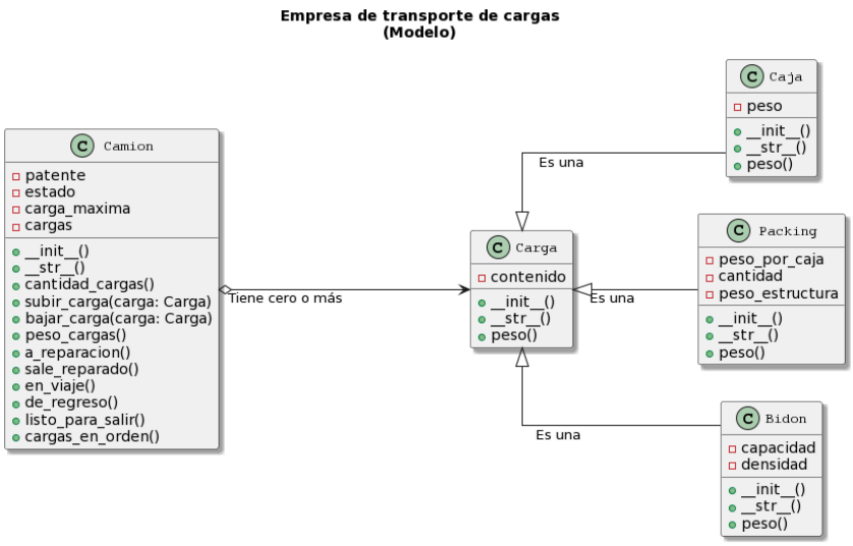
- Un **packing** es una estructura de madera que arriba tiene un montón de cajas; se envuelve todo con plástico para que no se desbanden las cajas. Todas las cajas tienen el mismo peso.
  - El peso de un packing es (peso de cada caja × cantidad de cajas) + peso de la estructura de madera que va abajo.
  - Para cada packing se informa qué llevan las cajas, p. ej. Material de construcción.
- De cada **caja suelta** se informa el peso individualmente, son todas distintas.
- El peso de un **bidón** es su capacidad en litros por la densidad (o sea, cuántos kg pesa un litro) del líquido que se le carga. Los bidones van siempre llenos hasta el tope.

Cada camión puede llevar hasta una carga máxima medida en kg. Además, cada camión puede: estar disponible para la carga (en cuyo caso ya puede tener cosas cargadas), estar en reparación, o estar de viaje.

#### Algunos requerimientos:

- Subir una carga al camión, donde la carga puede ser un packing, una caja suelta, o un bidón.
  - Considerar que no se puede saturar un camión con más peso de lo que su carga máxima permite.
- Bajar una carga del camión.
  - Siempre que el camión se encuentre disponible con cargas y que a su vez la carga se encuentre presente dentro de él.
- Conocer el total de cargas de un camión en todo momento y su peso.
- Permitir modificar el estado de un camión, sea porque:
  - Entró en reparación,
  - Salió de reparación,
  - Está en viaje, o
  - De regreso.
- Obtener el listado de cargas contenidas, como string, para imprimir por pantalla.
- Saber si un camión está listo para partir, que es:
  - Si está disponible para la carga,
  - Y el peso total de lo que tiene cargado es de al menos 75% de su carga máxima.

#### Modelo




Es el ejercicio integrador de la guía, ahora con tests. La validación de datos usa el mismo `utilities/Validador.py` de La fábrica.

Carga.py

```python
# Imports
from abc import ABC, abstractmethod

from utilities.Validador import Validador

# Exports
__all__: list[str] = ["Carga"]


class Carga(ABC):
    def __init__(self, contenido: str) -> None:
        self._contenido: str = Validador.esCadenaNoVacia(contenido)

    @property
    def contenido(self) -> str:
        return self._contenido

    @contenido.setter
    def contenido(self, contenido: str) -> None:
        self._contenido = Validador.esCadenaNoVacia(contenido)

    @abstractmethod
    def peso(self) -> float:
        pass

    def __str__(self) -> str:
        return f"Carga de {self.contenido} - Peso: {self.peso():.2f} kg"


if __name__ == "__main__":
    pass
```

Caja.py

```python
# Imports
from Carga import Carga
from utilities.Validador import Validador

# Exports
__all__: list[str] = ["Caja"]


class Caja(Carga):
    def __init__(self, contenido: str, peso: float) -> None:
        super().__init__(contenido)
        self._peso: float = Validador.esFlotantePositivo(peso)

    def peso(self) -> float:
        return self._peso

    def __str__(self) -> str:
        return f"Caja de {self.contenido} - Peso: {self.peso():.2f} kg"


if __name__ == "__main__":
    pass
```

Packing.py

```python
# Imports
from Carga import Carga
from utilities.Validador import Validador

# Exports
__all__: list[str] = ["Packing"]


class Packing(Carga):
    def __init__(
        self,
        contenido: str,
        peso_por_caja: float,
        cantidad: int,
        peso_estructura: float,
    ) -> None:
        super().__init__(contenido)
        self._peso_por_caja: float = Validador.esFlotantePositivo(peso_por_caja)
        self._cantidad: int = Validador.esEnteroPositivo(cantidad)
        self._peso_estructura: float = Validador.esFlotantePositivo(peso_estructura)

    @property
    def peso_por_caja(self) -> float:
        return self._peso_por_caja

    @peso_por_caja.setter
    def peso_por_caja(self, peso_por_caja: float) -> None:
        self._peso_por_caja = Validador.esFlotantePositivo(peso_por_caja)

    @property
    def cantidad(self) -> int:
        return self._cantidad

    @cantidad.setter
    def cantidad(self, cantidad: int) -> None:
        self._cantidad = Validador.esEnteroPositivo(cantidad)

    @property
    def peso_estructura(self) -> float:
        return self._peso_estructura

    @peso_estructura.setter
    def peso_estructura(self, peso_estructura: float) -> None:
        self._peso_estructura = Validador.esFlotantePositivo(peso_estructura)

    def peso(self) -> float:
        return (self.peso_por_caja * self.cantidad) + self.peso_estructura

    def __str__(self) -> str:
        return (
            f"Packing de {self.contenido} - {self.cantidad} cajas de "
            f"{self.peso_por_caja:.2f} kg c/u - "
            f"Estructura: {self.peso_estructura:.2f} kg - "
            f"Peso: {self.peso():.2f} kg"
        )


if __name__ == "__main__":
    pass
```

Bidon.py

```python
# Imports
from Carga import Carga
from utilities.Validador import Validador

# Exports
__all__: list[str] = ["Bidon"]


class Bidon(Carga):
    def __init__(self, contenido: str, capacidad: float, densidad: float) -> None:
        super().__init__(contenido)
        self._capacidad: float = Validador.esFlotantePositivo(capacidad)
        self._densidad: float = Validador.esFlotantePositivo(densidad)

    @property
    def capacidad(self) -> float:
        return self._capacidad

    @capacidad.setter
    def capacidad(self, capacidad: float) -> None:
        self._capacidad = Validador.esFlotantePositivo(capacidad)

    @property
    def densidad(self) -> float:
        return self._densidad

    @densidad.setter
    def densidad(self, densidad: float) -> None:
        self._densidad = Validador.esFlotantePositivo(densidad)

    def peso(self) -> float:
        return self.capacidad * self.densidad

    def __str__(self) -> str:
        return (
            f"Bidon de {self.contenido} - Capacidad: {self.capacidad:.2f} L - "
            f"Densidad: {self.densidad:.2f} kg/L - Peso: {self.peso():.2f} kg"
        )


if __name__ == "__main__":
    pass
```

Camion.py

```python
# El chequeo de carga en subir_carga es defensivo en tiempo de ejecucion
# aunque el parametro ya este tipado como Carga; por eso pyright lo marca
# como redundante.
# pyright: reportUnnecessaryIsInstance=false
# ruff: noqa: RUF012

# Imports
from Carga import Carga
from utilities.Validador import Validador

# Exports
__all__: list[str] = ["Camion"]
```

```python
class Camion:
    # Estados del enunciado: 1 disponible, 2 en reparacion, 3 de viaje
    ESTADOS: dict[int, str] = {1: "disponible", 2: "en reparacion", 3: "de viaje"}
    DISPONIBLE, REPARACION, VIAJE = 1, 2, 3

    PORCENTAJE_LISTO_PARA_SALIR: float = 0.75

    def __init__(self, patente: str, carga_maxima: float) -> None:
        self._patente: str = Validador.esCadenaNoVacia(patente)
        self._carga_maxima: float = Validador.esFlotantePositivo(carga_maxima)
        self._estado: int = Camion.DISPONIBLE
        self._cargas: list[Carga] = []

    @property
    def patente(self) -> str:
        return self._patente

    @patente.setter
    def patente(self, patente: str) -> None:
        self._patente = Validador.esCadenaNoVacia(patente)

    @property
    def carga_maxima(self) -> float:
        return self._carga_maxima

    @carga_maxima.setter
    def carga_maxima(self, carga_maxima: float) -> None:
        self._carga_maxima = Validador.esFlotantePositivo(carga_maxima)

    @property
    def estado(self) -> int:
        return self._estado

    @property
    def cargas(self) -> list[Carga]:
        return self._cargas

    def cantidad_cargas(self) -> int:
        return len(self.cargas)

    def peso_cargas(self) -> float:
        return sum(carga.peso() for carga in self.cargas)

    def subir_carga(self, carga: Carga) -> None:
        if not isinstance(carga, Carga):
            raise TypeError("Error el valor ingresado no corresponde a una carga")
        if self.peso_cargas() + carga.peso() > self.carga_maxima:
            raise ValueError("Error no se puede superar la carga maxima del camion")
        self._cargas.append(carga)

    def bajar_carga(self, carga: Carga) -> None:
        if self.estado != Camion.DISPONIBLE or carga not in self.cargas:
            raise ValueError("Error no se puede bajar la carga del camion")
        self._cargas.remove(carga)

    def a_reparacion(self) -> None:
        self._estado = Camion.REPARACION

    def sale_reparado(self) -> None:
        self._estado = Camion.DISPONIBLE
```

```
    def en_viaje(self) -> None:
        self._estado = Camion.VIAJE

    def de_regreso(self) -> None:
        self._estado = Camion.DISPONIBLE

    def listo_para_salir(self) -> bool:
        return (
            self.estado == Camion.DISPONIBLE
            and self.peso_cargas()
            >= Camion.PORCENTAJE_LISTO_PARA_SALIR * self.carga_maxima
        )

    def cargas_en_orden(self) -> str:
        return "\n".join(str(carga) for carga in self.cargas)

    def __str__(self) -> str:
        return (
            f"Camion {self.patente} - Estado: {Camion.ESTADOS[self.estado]} - "
            f"Cargas: {self.cantidad_cargas()} - "
            f"Peso: {self.peso_cargas():.2f}/{self.carga_maxima:.2f} kg"
        )


if __name__ == "__main__":
    pass
```

main.py

```
"""Carga los packings, cajas y bidones desde los archivos CSV de data/."""

import csv
from pathlib import Path

from Bidon import Bidon
from Caja import Caja
from Camion import Camion
from Carga import Carga
from Packing import Packing

DIRECTORIO_DATOS: Path = Path(__file__).parent / "data"
ARCHIVO_BIDONES: Path = DIRECTORIO_DATOS / "bidones.csv"
ARCHIVO_CAJAS: Path = DIRECTORIO_DATOS / "cajas.csv"
ARCHIVO_PACKING: Path = DIRECTORIO_DATOS / "packing.csv"

CARGA_MAXIMA_CAMION: float = 200.0


def leer_csv(ruta: Path) -> list[dict[str, str]]:
    with open(ruta, newline="", encoding="utf-8") as archivo:
        return list(csv.DictReader(archivo))


def cargar_bidones(ruta: Path = ARCHIVO_BIDONES) -> list[Bidon]:
    return [
        Bidon(
            contenido=fila["producto"],
            capacidad=float(fila["capacidad"]),
            densidad=float(fila["densidad"]),
        )
        for fila in leer_csv(ruta)
```

```
    ]


def cargar_cajas(ruta: Path = ARCHIVO_CAJAS) -> list[Caja]:
    return [
        Caja(contenido=fila["producto"], peso=float(fila["peso"]))
        for fila in leer_csv(ruta)
    ]


def cargar_packings(ruta: Path = ARCHIVO_PACKING) -> list[Packing]:
    return [
        Packing(
            contenido=fila["producto"],
            peso_por_caja=float(fila["peso"]),
            cantidad=int(fila["cantidad"]),
            peso_estructura=float(fila["estructura"]),
        )
        for fila in leer_csv(ruta)
    ]


def cargar_todas_las_cargas() -> list[Carga]:
    cargas: list[Carga] = []
    cargas.extend(cargar_bidones())
    cargas.extend(cargar_cajas())
    cargas.extend(cargar_packings())
    return cargas


def crear_camiones() -> list[Camion]:
    return [
        Camion("AA001AA", CARGA_MAXIMA_CAMION),
        Camion("BB002BB", CARGA_MAXIMA_CAMION),
        Camion("CC003CC", CARGA_MAXIMA_CAMION),
    ]


def distribuir_cargas(camiones: list[Camion], cargas: list[Carga]) -> None:
    """Sube cada carga al primer camion de la lista donde todavia entre."""
    for carga in cargas:
        for camion in camiones:
            if camion.peso_cargas() + carga.peso() <= camion.carga_maxima:
                camion.subir_carga(carga)
                break


def main() -> None:
    camiones: list[Camion] = crear_camiones()
    cargas: list[Carga] = cargar_todas_las_cargas()
    distribuir_cargas(camiones, cargas)

    print(f"Cargas a distribuir: {len(cargas)}")
    print()

    for camion in camiones:
        print(camion)
        print(f"  Listo para salir: {'si' if camion.listo_para_salir() else 'no'}")
    print()

    primer_camion: Camion = camiones[0]
```

```
    print(f"Detalle de cargas de {primer_camion.patente}:")
    print(primer_camion.cargas_en_orden())
    print()

    if primer_camion.cargas:
        primera_carga: Carga = primer_camion.cargas[0]
        primer_camion.bajar_carga(primera_carga)
        print(
            f"Se bajo una carga de {primer_camion.patente}. "
            f"Cargas restantes: {primer_camion.cantidad_cargas()}"
        )
    print()

    primer_camion.a_reparacion()
    print(
        f"{primer_camion.patente} entro en reparacion - Estado: {primer_camion.estado}"
    )
    primer_camion.sale_reparado()
    print(
        f"{primer_camion.patente} salio de reparacion - Estado: {primer_camion.estado}"
    )

    segundo_camion: Camion = camiones[1]
    segundo_camion.en_viaje()
    print(f"{segundo_camion.patente} esta en viaje - Estado: {segundo_camion.estado}")
    segundo_camion.de_regreso()
    print(f"{segundo_camion.patente} esta de regreso - Estado: {segundo_camion.estado}")


if __name__ == "__main__":
    main()
```

✓ Pasa los **36** tests de la cátedra (`tests/autocontenido/test_empresa_autocontenido.py`) · en Linux, con una copia en minúscula de `Camion.py` y `Carga.py`

Salida al correrlo

```
Cargas a distribuir: 50

Camion AA001AA - Estado: disponible - Cargas: 35 - Peso: 197.57/200.00 kg
  Listo para salir: si
Camion BB002BB - Estado: disponible - Cargas: 14 - Peso: 188.00/200.00 kg
  Listo para salir: si
Camion CC003CC - Estado: disponible - Cargas: 1 - Peso: 18.00/200.00 kg
  Listo para salir: no

Detalle de cargas de AA001AA:
Bidon de Sopa en lata - Capacidad: 2.00 L - Densidad: 1.78 kg/L - Peso: 3.56 kg
Bidon de Café - Capacidad: 2.00 L - Densidad: 1.90 kg/L - Peso: 3.80 kg
Bidon de Sopa en lata - Capacidad: 5.00 L - Densidad: 1.03 kg/L - Peso: 5.15 kg
Bidon de Sopa en lata - Capacidad: 1.00 L - Densidad: 1.06 kg/L - Peso: 1.06 kg
Bidon de Aceite de oliva - Capacidad: 5.00 L - Densidad: 1.34 kg/L - Peso: 6.70 kg
Bidon de Refresco de cola - Capacidad: 4.00 L - Densidad: 1.31 kg/L - Peso: 5.24 kg
Bidon de Vino tinto - Capacidad: 4.00 L - Densidad: 1.24 kg/L - Peso: 4.96 kg
Bidon de Refresco de cola - Capacidad: 1.00 L - Densidad: 1.93 kg/L - Peso: 1.93 kg
Bidon de Jugo de naranja - Capacidad: 4.00 L - Densidad: 1.38 kg/L - Peso: 5.52 kg
Bidon de Aceite de oliva - Capacidad: 3.00 L - Densidad: 1.65 kg/L - Peso: 4.95 kg
Bidon de Aceite de oliva - Capacidad: 3.00 L - Densidad: 1.14 kg/L - Peso: 3.42 kg
Bidon de Té - Capacidad: 5.00 L - Densidad: 1.30 kg/L - Peso: 6.50 kg
Bidon de Jugo de naranja - Capacidad: 5.00 L - Densidad: 1.35 kg/L - Peso: 6.75 kg
Bidon de Cerveza - Capacidad: 3.00 L - Densidad: 1.11 kg/L - Peso: 3.33 kg
… (22 renglones más)
Se bajo una carga de AA001AA. Cargas restantes: 34

AA001AA entro en reparacion - Estado: 2
AA001AA salio de reparacion - Estado: 1
BB002BB esta en viaje - Estado: 3
BB002BB esta de regreso - Estado: 1
```

# 5 Modelo de parcial · La inmobiliaria

Modelo de parcial (repositorio) | Resolución de la cátedra

### Enunciado (tal cual)

Una inmobiliaria de la ciudad administra una serie de inmuebles disponibles para ser alquilados.

De cada inmueble se conoce un código numérico, el nombre del propietario, la superficie de la construcción y el importe base del alquiler mensual.

El importe base es asignado por la inmobiliaria en función de las condiciones del mercado, pero el importe definitivo es calculado a partir del importe base y de las características particulares del inmueble.

- En el caso de las **casas**, al importe base se le adicionan $30.000 por cada dormitorio que posea y $100.000 si posee pileta de natación.
- Los **departamentos** ven incrementado su importe base en $20.000 si se encuentran en un piso inferior al tercero y se les incorpora el importe de las expensas.

Se necesita un programa que lea del archivo `inmuebles.csv` la lista de todos los inmuebles y los almacene en algún objeto (no en la función principal) que ofrezca los siguientes métodos:

1. **Suma de alquileres**: informe el total a recaudar en concepto de alquileres si todos los inmuebles se encuentran alquilados.
2. **Cantidad de casas premium**: informe la cantidad de casas de más de 150 metros cuadrados, más de 2 dormitorios y que posean pileta.
3. **Propietario del alquiler más bajo**: informe el nombre del propietario del departamento con el alquiler cuyo importe definitivo sea el más bajo.

La función principal debe ingresar los datos desde el archivo de texto y finalizar luego de imprimir el resultado de la ejecución de los tres métodos anteriores.

### Estructura del archivo `inmuebles.csv`

El archivo posee la siguiente estructura (sin línea de títulos, una línea por inmueble):

1. Tipo de inmueble: 1 para casas y 2 para departamentos
2. Código: número sin repetición que identifica cada inmueble
3. Nombre del propietario
4. Alquiler base: número de tipo float
5. Superficie: número entero expresado en metros cuadrados
6. Si es una casa, la cantidad de habitaciones; si es un departamento, el importe de las expensas
7. Si es una casa, un número 1 si tiene pileta y 0 si no tiene; si es un departamento, un número indicando el piso, con 0 para planta baja

### Contrato esperado (para los tests)

Las clases deben llamarse y comportarse de la siguiente manera para que pasen las pruebas:

- `Inmueble(codigo, propietario, superficie, alquiler_base)` — clase base con atributos homónimos y método `alquiler()`.
- `Casa(codigo, propietario, superficie, alquiler_base, dormitorios, pileta)` — `pileta` booleano; redefine `alquiler()`.
- `Departamento(codigo, propietario, superficie, alquiler_base, expensas, piso)` — redefine `alquiler()`.

- `Inmobiliaria()` — contiene la colección de inmuebles y ofrece:
  - `agregar(inmueble)`
  - `suma_alquileres()`
  - `cantidad_casas_premium()`
  - `propietario_alquiler_mas_bajo()`

inmueble.py

```
from abc import ABC, abstractmethod

class Inmueble(ABC):

    def __init__(self, codigo, propietario, superficie, importe_base):
        self.codigo = codigo
        self.propietario = propietario
        self.superficie = superficie
        self.importe_base = importe_base


    @abstractmethod
    def alquiler(self):
        pass
```

casa.py

```
from inmueble import Inmueble

class Casa(Inmueble):

    def __init__(self, codigo, propietario, superficie, importe_base, dormitorios, tiene_pileta):
        super().__init__(codigo, propietario, superficie, importe_base)
        self.dormitorios = dormitorios
        self.tiene_pileta = tiene_pileta


    def alquiler(self):
        importe_dormitorios = self.dormitorios * 30000
        importe_pileta = 0
        if self.tiene_pileta: importe_pileta = 100000

        return self.importe_base + importe_dormitorios + importe_pileta
```

departamento.py

```
from inmueble import Inmueble

class Departamento(Inmueble):

    def __init__(self, codigo, propietario, superficie, importe_base, importe_expensas, piso):
        super().__init__(codigo, propietario, superficie, importe_base)
        self.importe_expensas = importe_expensas
        self.piso = piso


    def alquiler(self):
        importe_piso = 0
        if self.piso < 3: importe_piso = 20000

        return self.importe_base + importe_piso + self.importe_expensas
```

inmobiliaria.py

```
from casa import Casa
from departamento import Departamento

class Inmobiliaria:

    def __init__(self):
        self.inmuebles = []

    def agregar(self, inmueble):
        self.inmuebles.append(inmueble)

    def suma_alquileres(self):

        return sum(list(map(lambda i: i.alquiler(), self.inmuebles)))


    def cantidad_casas_premium(self):

        c = 0
        for inm in self.inmuebles:
            if isinstance(inm, Casa) and inm.superficie > 150 and inm.dormitorios > 2 and
inm.tiene_pileta:
                c += 1

        return c

    def propietario_alquiler_mas_bajo(self):

        menor = None
        for inm in self.inmuebles:
            if isinstance(inm, Departamento):
                if menor is None or inm.alquiler() < menor.alquiler():
                    menor = inm

        if menor is None: return None
        return menor.propietario
```

main.py

```
from casa import Casa
from departamento import Departamento
from inmobiliaria import Inmobiliaria

def main():

    inm = Inmobiliaria()
    archivo = open("inmuebles.csv")
    for linea in archivo:
        datos = linea.split(",")
        tipo = int(datos[0])
        codigo = int(datos[1])
        propietario = datos[2]
        importe_base = float(datos[3])
        superficie = int(datos[4])

        if tipo == 1:
            dormitorios = int(datos[5])
            tiene_pileta = int(datos[6]) == 1
            casa = Casa(codigo, propietario, superficie, importe_base, dormitorios, tiene_pileta)
            inm.agregar(casa)
        else:
            expensas = float(datos[5])
            piso = int(datos[6])
            depto = Departamento(codigo, propietario, superficie, importe_base, expensas, piso)
            inm.agregar(depto)


    archivo.close()

    print(inm.suma_alquileres())
    print(inm.cantidad_casas_premium())
    print(inm.propietario_alquiler_mas_bajo())



if __name__ == "__main__":
    main()
```

✓ Pasa los **16** tests de la cátedra ( `test_inmobiliaria.py` )

Salida al correr main.py (suma de alquileres, casas premium y propietario del alquiler más bajo)

```
78832240.0
11
MARIA JOSE ESTEBAN
```

# 6 Modelo de parcial · El supermercado

Modelo de parcial (repositorio) Solución propuesta (verificada con los tests)

### Enunciado (tal cual)

Una cadena de supermercados posee varias sucursales con diferentes características, agrupadas en tres formatos diferentes.

- **Hipermercados**: además del punto de venta, poseen boxes que cede a comerciantes pequeños a cambio de un alquiler mensual.
- **Supermercados tradicionales**: en edificaciones propias; algunos tienen un formato más orientado a la venta mayorista.
- **Mini**: en locales que alquila en plantas bajas de edificios de viviendas, abonando un alquiler mensual al consorcio.

De cada sucursal el supermercado registra la superficie en metros cuadrados del salón de ventas y el total facturado en el último mes. Adicionalmente existen dos cálculos relevantes: el **resultado comercial** y el **índice de rentabilidad**.

- El resultado comercial de cada sucursal tipo supermercado es equivalente a la facturación. En los locales de hipermercados se suma el total ganado en concepto de alquileres; en los de modalidad Mini se resta el alquiler abonado al edificio.
- El índice de rentabilidad es el cociente entre el resultado comercial y la superficie. Un local de hipermercado es rentable si su índice es mayor a 50; uno Mini si es mayor a 35. Los locales tradicionales son rentables si su índice es mayor a 40, pero los mayoristas deben tener un índice mayor a 45.

Se necesita un programa que lea del archivo `sucursales.csv` la lista de todas las sucursales y las almacene en algún objeto (no en la función principal) que ofrezca los siguientes métodos:

1. **Suma de ganancia**: total del resultado comercial de todas las sucursales.
2. **Cantidad de locales no rentables**: cantidad de sucursales cuyo índice de rentabilidad sea menor al exigido.
3. **Local más rentable**: número y tipo de la sucursal cuyo índice de rentabilidad sea el mayor de todos.

La función principal debe ingresar los datos desde el archivo de texto y finalizar luego de imprimir el resultado de la ejecución de los tres métodos anteriores.

### Estructura del archivo `sucursales.csv`

El archivo posee la siguiente estructura (sin línea de títulos, una línea por sucursal):

1. Tipo de sucursal: 1 para hiper, 2 para super y 3 para mini
2. Número: número sin repetición que identifica cada sucursal
3. Superficie: número de tipo entero con la superficie en metros cuadrados
4. Facturación: número de tipo float con la facturación del local, expresada en miles de pesos
5. Si es una sucursal de hipermercado, el importe ganado por alquileres; si es de supermercado, un 1 si es mayorista y 0 si no lo es; si es de modalidad Mini, el importe pagado por el alquiler

### Contrato esperado (para los tests)

Las clases deben llamarse y comportarse de la siguiente manera para que pasen las pruebas:

- `Sucursal(numero, superficie, facturacion)` — clase base con atributos homónimos y métodos `resultado_comercial()`, `es_rentable()` y `tipo()`.

- `Hiper(numero, superficie, facturacion, alquileres)` — suma los alquileres al resultado; rentable si índice > 50; `tipo()` = 1.
- `Super(numero, superficie, facturacion, mayorista)` — el resultado es la facturación; rentable si índice > 40 (tradicional) o > 45 (mayorista); `tipo()` = 2.
- `Mini(numero, superficie, facturacion, alquiler)` — resta el alquiler al resultado; rentable si índice > 35; `tipo()` = 3.
- `Empresa()` — contiene la colección de sucursales y ofrece:
  - `agregar_sucursal(sucursal)`
  - `suma_ganancia()`
  - `cantidad_no_rentables()`
  - `local_mas_rentable()`

**Cómo se piensa**

- Lo que cambia por tipo es el resultado comercial y el índice mínimo: cada subclase implementa `resultado_comercial()` e `indice_minimo()`, y la base calcula `indice_rentabilidad()` y `es_rentable()` una sola vez.
- El test pide `str(local) == "1 1"`: número y tipo separados por un espacio.

sucursal.py

```python
from abc import ABC, abstractmethod


class Sucursal(ABC):
    def __init__(self, numero, superficie, facturacion):
        self.numero = numero
        self.superficie = superficie
        self.facturacion = facturacion

    @abstractmethod
    def resultado_comercial(self):
        """Cada formato lo calcula distinto."""

    @abstractmethod
    def indice_minimo(self):
        """El índice que tiene que superar para ser rentable."""

    @abstractmethod
    def tipo(self):
        """1: hiper, 2: super, 3: mini"""

    def indice_rentabilidad(self):
        return self.resultado_comercial() / self.superficie

    def es_rentable(self):
        return self.indice_rentabilidad() > self.indice_minimo()

    def __str__(self):
        return f"{self.numero} {self.tipo()}"          # número y tipo, como pide el test
```

hiper.py

```python
from sucursal import Sucursal


class Hiper(Sucursal):
    def __init__(self, numero, superficie, facturacion, alquileres):
        super().__init__(numero, superficie, facturacion)
        self.alquileres = alquileres            # lo que cobra por los boxes

    def resultado_comercial(self):
        return self.facturacion + self.alquileres

    def indice_minimo(self):
        return 50

    def tipo(self):
        return 1
```

super.py

```python
from sucursal import Sucursal


class Super(Sucursal):
    def __init__(self, numero, superficie, facturacion, mayorista):
        super().__init__(numero, superficie, facturacion)
        self.mayorista = mayorista

    def resultado_comercial(self):
        return self.facturacion

    def indice_minimo(self):
        return 45 if self.mayorista else 40

    def tipo(self):
        return 2
```

mini.py

```python
from sucursal import Sucursal


class Mini(Sucursal):
    def __init__(self, numero, superficie, facturacion, alquiler):
        super().__init__(numero, superficie, facturacion)
        self.alquiler = alquiler                # lo que paga al consorcio

    def resultado_comercial(self):
        return self.facturacion - self.alquiler

    def indice_minimo(self):
        return 35

    def tipo(self):
        return 3
```

empresa.py

```
class Empresa:
    def __init__(self):
        self.sucursales = []

    def agregar_sucursal(self, sucursal):
        self.sucursales.append(sucursal)

    def suma_ganancia(self):
        return sum(s.resultado_comercial() for s in self.sucursales)

    def cantidad_no_rentables(self):
        return len([s for s in self.sucursales if not s.es_rentable()])

    def local_mas_rentable(self):
        return max(self.sucursales, key=lambda s: s.indice_rentabilidad(), default=None)
```

main.py

```
from empresa import Empresa
from hiper import Hiper
from mini import Mini
from super import Super

TIPOS = {1: "hiper", 2: "super", 3: "mini"}


def leer_sucursales(nombre_archivo, empresa):
    with open(nombre_archivo, encoding="utf-8") as archivo:
        for linea in archivo:
            if not linea.strip():
                continue
            tipo, numero, superficie, facturacion, extra = linea.strip().split(",")
            numero, superficie, facturacion = int(numero), int(superficie), float(facturacion)
            if tipo == "1":
                empresa.agregar_sucursal(Hiper(numero, superficie, facturacion, float(extra)))
            elif tipo == "2":
                empresa.agregar_sucursal(Super(numero, superficie, facturacion, extra == "1"))
            else:
                empresa.agregar_sucursal(Mini(numero, superficie, facturacion, float(extra)))


def main():
    empresa = Empresa()
    leer_sucursales("sucursales.csv", empresa)
    print("Sucursales leídas:", len(empresa.sucursales))
    print(f"Suma de ganancia: {empresa.suma_ganancia():,.2f} (miles de pesos)")
    print("Locales no rentables:", empresa.cantidad_no_rentables())
    local = empresa.local_mas_rentable()
    print(f"Local más rentable: número {local.numero}, tipo {local.tipo()}
({TIPOS[local.tipo()]}), "
          f"índice {local.indice_rentabilidad():.2f}")


if __name__ == "__main__":
    main()
```

✓ Pasa los **13** tests de la cátedra ( `test_supermercado.py` )

Salida al correrlo

```
Sucursales leídas: 199
Suma de ganancia: -5,602,000.00 (miles de pesos)
Locales no rentables: 112
Local más rentable: número 159, tipo 1 (hiper), índice 1310.82
```

**Para tener en cuenta**

- La suma de ganancia da **negativa** por los datos, no por un error: en el CSV, 70 de los 72 locales Mini pagan de alquiler más de lo que facturan. Se verificó recalculando todo por separado.